SCAN-CONVERTING A CIRCLE

A circle is a symmetrical figure.

Using 8-way symmetry, one calculated point on a circle generates seven additional points by reflecting across axes and diagonals, reducing computation.

$$P_1 = (x, y) \qquad P_5 = (-x, -y)$$
$$P_2 = (y, x) \qquad P_6 = (-y, -x)$$
$$P_3 = (-y, x) \qquad P_7 = (y, -x)$$
$$P_4 = (-x, y) \qquad P_8 = (x, -y)$$

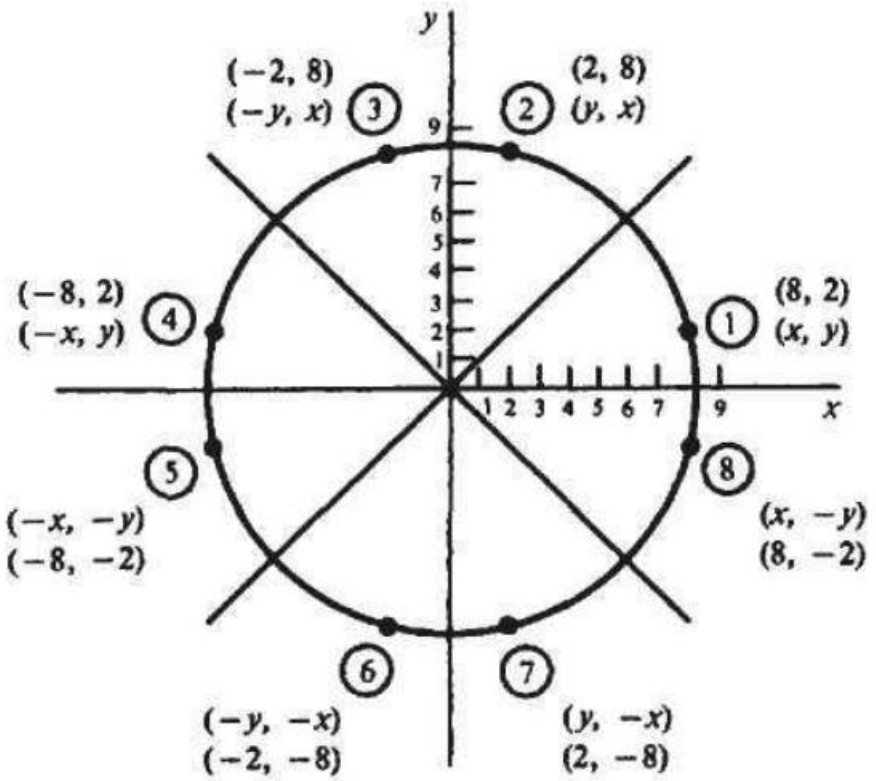


Fig. 3-4 Eight-way symmetry of a circle.

Defining a circle.

The first method defines a circle with the second-order polynomial equation (see Fig. 3-5)

$$y^2 = r^2 - x^2$$

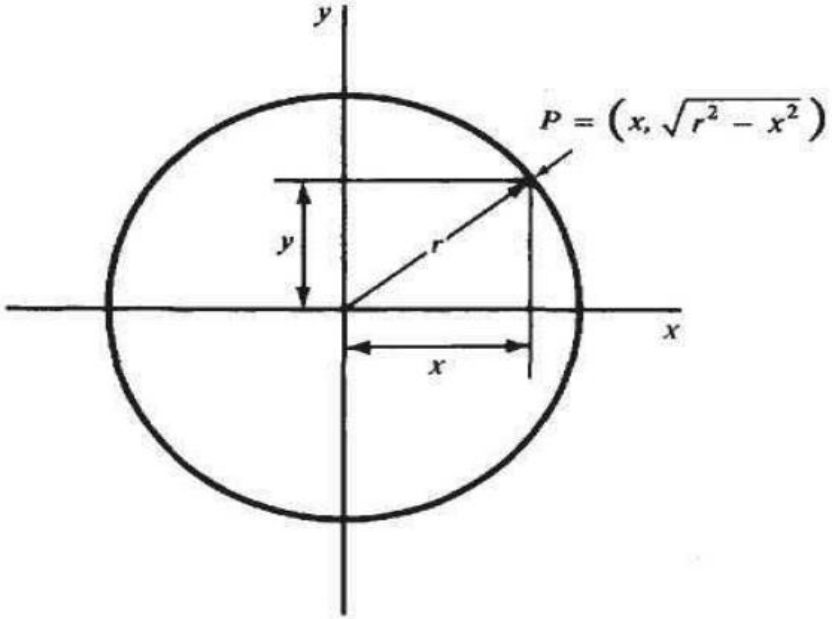


Fig. 3-5 Circle defined with a second-degree polynomial equation.

The second method of defining a circle makes use of trigonometric functions (see Fig. 3-6):

$$x = r \cos \Theta$$
$$y = r \sin \Theta$$

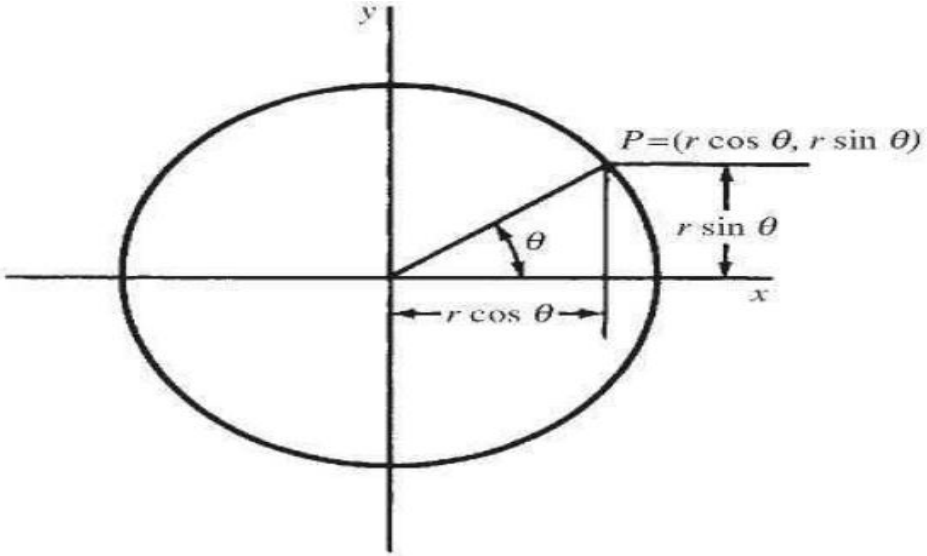


Fig. 3-6 Circle defined with trigonometric functions.

Bresenham's Circle Algorithm

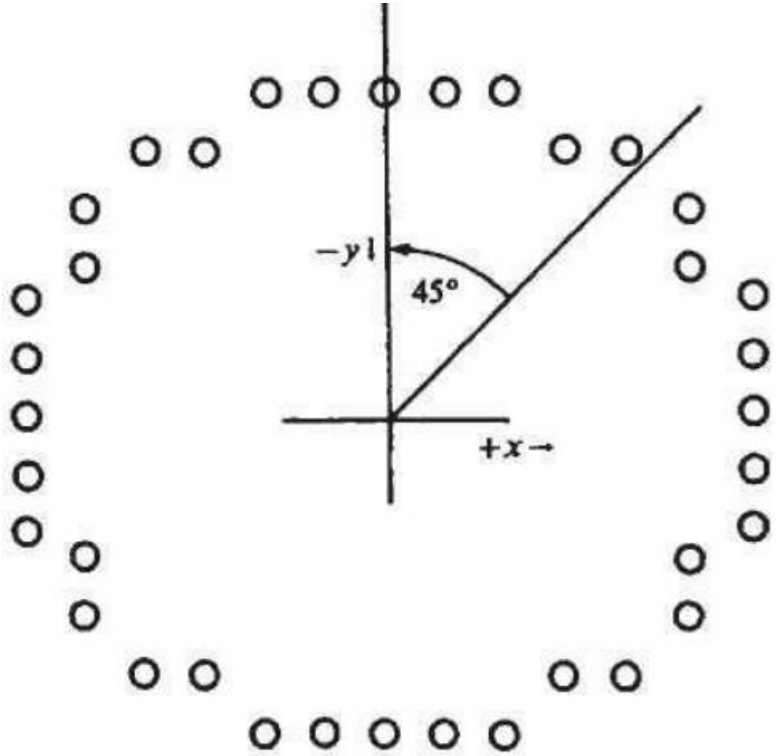
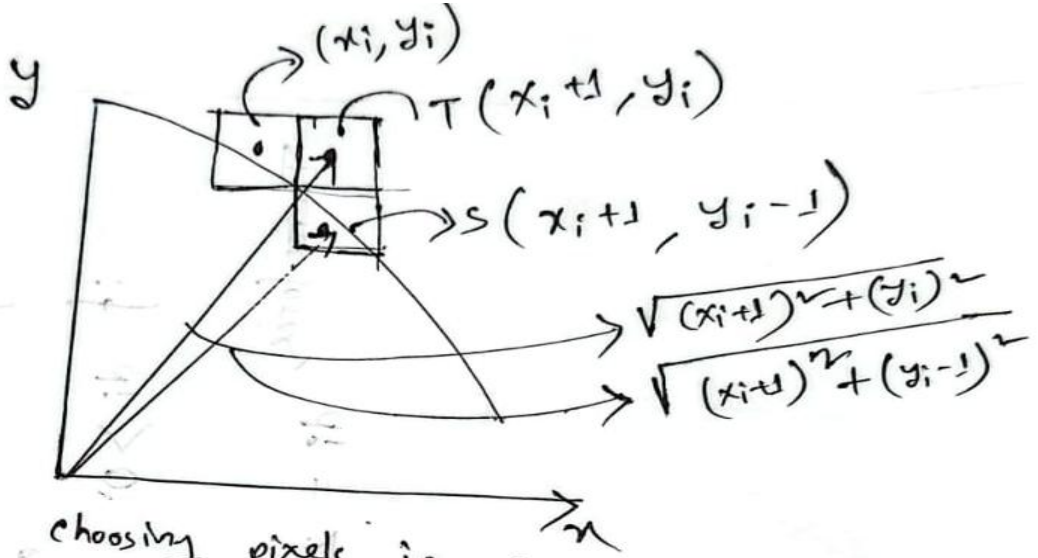


Fig. 3-7 Circle scan-converted with Bresenham's algorithm.

Bresenham's circle algorithm generates points only from 90° to 45°, choosing between two pixel moves to best approximate the true circle.

Bresenham's Circle Algorithm



choosing pixels in Bresenham's circle algorithm

At each step from current point (x_i, y_i), the next x is two candidate,

$$x_{i+1} = x_i + 1$$

$$T(x_i+1, y_i)$$

$$S(x_i+1, y_i-1)$$

now, Defining a relative distance from a point to the circle

$$D(x,y) = x^2 + y^2 - r^2$$

if $D<0$, pixel lies inside the circle

if $D>0$, pixel lies outside the circle

ned with CamScanner

Bresenham's Circle Algorithm

now, for T & S,

$$D(T) = (x_i+1)^2 + y_i^2 - r^2$$

$$D(S) = (x_i+1)^2 + (y_i-1)^2 - r^2$$

now, define a decision variable

$$d_i = D(T) + D(S)$$

if $d_i < 0$ means $|D(T)| < |D(S)|$

choose T.

if $d_i > 0$ means $|D(T)| > |D(S)|$

choose S.

Therefore,

$$d_i = 2(x_i+1)^2 + y_i^2 + (y_i-1)^2 - r^2$$

- r^2

Bresenham's Circle Algorithm

for choosing next step
decision variable

$$d_{i+1} = 2(x_{i+1}+1)^2 + y_{i+1}^2 + (y_{i+1}-1)^2 - 2r^2$$

Hence

$$d_{i+1} - d_i = 2(x_{i+1}+1)^2 + y_{i+1}^2 + (y_{i+1}-1)^2 - 2(x_i+1)^2 - y_i^2 - (y_i-1)^2$$

We know very well that

$$x_{i+1} = x_i + 1$$

$$= 2(x_i+2)^2 + y_{i+1}^2 + (y_{i+1}-1)^2 - 2(x_i+1)^2 - y_i^2 - (y_i-1)^2$$

$$= \cancel{2x_i^2 + 2}$$

$$= 2(x_i^2 + 4x_i + 4) + y_{i+1}^2 + y_{i+1}^2 - 2y_{i+1} + 1 - 2(x_i^2 + 2x_i + 1) - y_i^2 - (y_i^2 - 2y_i + 1)$$

$$= \cancel{2(x_i}$$

$$= 2x_i^2 + 8x_i + 8 + y_{i+1}^2 + y_{i+1}^2 - 2y_{i+1} + 4 - 2x_i^2 - 4x_i - 2 - y_i^2 - y_i^2 + 2y_i - 1$$

$$= 4x_i + 2(y_{i+1}^2 - y_i^2) - 2(y_{i+1} - y_i) + 6$$

Bresenham's Circle Algorithm

$$d_{i+1} = d_i + 4x_i + 2(y_{i+1}^2 - y_i^2) - 2(y_{i+1} - y_i) + 6$$

if $d_i < 0$,

$y_{i+1} = y_i$, so $d_{i+1} = d_i + 4x_i + 6$

if $d_i > 0$

$y_{i+1} = y_i - 1$

$d_{i+1} = d_i + 4(x_i - y_i) + 10$

for starting pixel $(0, r)$

$d_i = 2(x_i+1)^2 + y_i^2 + (y_i - 1)^2 - 2r^2$

$x_i = 0$, $y_i = r$,

$d_i = 2 \times (0+1)^2 + r^2 + (r-1)^2 - 2r^2$

$= 2 + r^2 + r^2 - 2r + 1 - 2r^2$

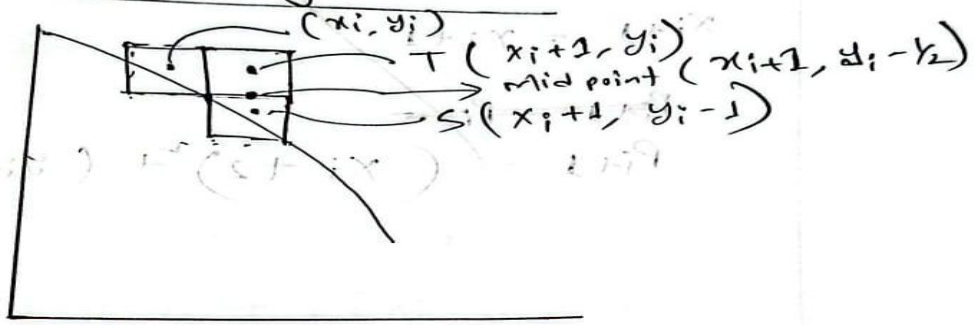
$= 3 - 2r$

Bresenham's Circle Algorithm

so, the derived algorithm (Pseudocode)

```
x = 0, y = r, d = 3 - 2r
while (x <= y) {
    setPixel(x, y);
    if (d < 0)
        d = d + 4·x + 6;
    else {
        d = d + 4(x - y) + 10;
        y--;
    }
    x++;
}
```

Deriving Midpoint Circle Algorithm:



decision parameter,

$$P_i = f(x_i+1, y_i-\tfrac{1}{2}) = (x_i+1)^2 + (y_i-\tfrac{1}{2})^2 - r^2$$

if $d<0$, mid point inside circle $\rightarrow$ choose T.

if $d \geq 0$, mid point outside circle $\rightarrow$ choose S.

$$P_{i+1} = (x_{i+1}+1)^2 + (y_{i+1}-\tfrac{1}{2})^2 - r^2$$

$x_{i+1} = x_i + 1$

$$P_{i+1} = (x_i+2)^2 + \cancel{(y_i+\tfrac{1}{2})^2 - r^2}$$
$$(y_{i+1}-\tfrac{1}{2})^2 - r^2$$

$$P_{i+1} - P_i = (x_i+2)^2 - (x_i+1)^2 + (y_{i+1}-\tfrac{1}{2})^2 - (y_i-\tfrac{1}{2})^2$$

$$x_i^2 + 4x_i + 4 - x^2 - 2x_i - 1 + y_{i+1}^2 - y_{i+1} + \tfrac{1}{4} - y_i^2 + y_i - \tfrac{1}{4}$$

$$P_{i+1} = P_i + 2(x_i+1) + 1 + (y_{i+1}^2 - y_i^2) - (y_{i+1} - y_i)$$

if $P_i < 0$, ~~y_{i+1}~~ $y_{i+1} = y_i$

$$P_{i+1} = P_i + 2x_i + 3$$

if $P_i \geq 0$ $\quad y_{i+1} = y_i - 1$

$$P_{i+1} = P_i + 2(x_i - y_i) + 5$$

for initial decision value, for $(0, r)$

$$P_i = (0+1)^2 + (r - 1/2)^2 - r^2$$

$$= \frac{5}{4} - r$$

$\longrightarrow$ fraction

for removing fraction.

$5/4 \rightarrow 1$

$$P_i = 1 - r$$

Midpoint Circle Algorithm

```
x=0, y=r, p=1-r
while(x<=y){
    setPixel(x,y);
    if(p<0){ p=p+2x+3; }
    else { p=p+2(x-y)+5;
           y--; }
    x++;
}
```